Chapter 5 – Pre-trained Language Models, LLMs and Financial Applications

Advanced Machine Learning for Finance

Giovanni Della Lunga

University of Bologna

April-May 2026

Introduction

Where We Are in the Course

Chapter 1 – Classical NLP
Bag-of-words, TF-IDF, BM25: sparse, high-dimensional, no word-order, no semantics.

Chapters 2 & 3 – Embeddings and Sequences
Word2Vec / GloVe: dense but *static*. RNNs, LSTMs, encoder-decoder: sequential processing, attention seed.

Chapter 4 – Attention and the Transformer
Scaled dot-product attention, multi-head attention, positional encoding, full encoder-decoder stack. Encoder-only vs. decoder-only split.

Today's question

The transformer is the architecture.
Pre-training sets its parameters.
Fine-tuning adapts it to a task.
Retrieval extends its memory.

When does the result add value in a financial workflow – and when does it not?

Why This Chapter Matters

Conceptual reason

The pre-training paradigm is the single most important shift in applied NLP of the last decade. Without understanding it, LLM applications remain black boxes and fail in production.

Practical reason

Every financial NLP pipeline built today – sentiment classifiers, document extractors, RAG question-answering systems – sits on top of one of the models described in this chapter.

Honest caveat

Understanding *how* these models work is also what allows you to know **when not to use them**. That is the most important skill in applied machine learning.

Learning Objectives

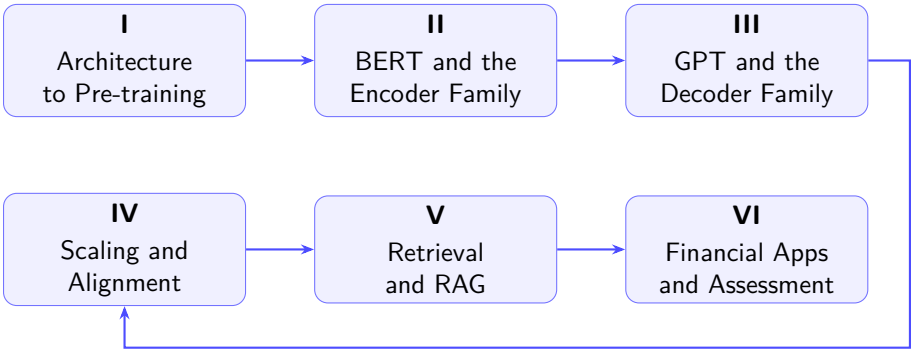
By the end of this chapter, students should be able to:

- explain the MLM and autoregressive pre-training objectives and connect them to the encoder-only / decoder-only split;
- describe the fine-tuning paradigm and identify the role of the `[CLS]` token for sentence-level classification;
- trace the GPT genealogy and explain why scale produces qualitatively new behaviour;
- formalise RLHF and explain why alignment carries direct financial and legal implications;
- construct a RAG pipeline and identify its principal failure modes;
- apply a **benchmarking discipline** to financial NLP tasks: always measure against the simplest reasonable alternative.

Guiding question

Given a trained transformer, how do we use it – and how do we decide whether to use it at all?

Chapter Roadmap



From Architecture to Pre-training

The Architecture Is Not the Model

Chapter 4 described the transformer as a parametric function:

$$f_\theta \ : \ \text{token IDs} \longrightarrow \text{contextual representations}$$

with hundreds of millions of parameters θ.

The architecture specifies only the functional form. The parameters are determined entirely by the training procedure.

Two separate questions

- *Architecture:* how are tokens related inside the forward pass? $\Rightarrow$ answered in Chapters 3 and 4.
- *Training:* how are the parameters set? $\Rightarrow$ the subject of this chapter.

Three Paradigms from One Architecture

	Encoder-only	**Decoder-only**	**Encoder-decoder**
Canonical model	BERT	GPT	T5 / BART
Pre-training obj.	Masked LM	Autoregressive LM	Seq2seq denoising
Attention	Bidirectional	Causal masked	Mixed
Token sees	Full context	Past tokens only	Depends on role
Main use	Classify, extract	Generate, prompt	Translate, summarise

Key insight

All three families use the same scaled dot-product attention from Chapter 4. The difference is *which positions can attend to which* – controlled by the attention mask.

Pre-training as Self-supervised Learning

Core idea: train on vast amounts of *raw, unlabelled text* by asking the model to predict something about the input itself. No human annotation is required – the labels come from the data.

Encoder: fill the gap (MLM)

"The `[MASK]` rose sharply after the announcement."
$\Rightarrow$ predict *yield*.

Decoder: predict next token

"The yield rose sharply after the. . . "
$\Rightarrow$ predict *announcement*.

Connection to Chapter 2

Word2Vec also learned from self-supervision, but was shallow and task-free. Pre-training a deep transformer on billions of tokens produces representations that transfer across a wide range of downstream tasks.

Why This Paradigm Changed Everything

Before pre-training

- Every task needed a large labelled dataset.
- Features were hand-engineered or trained from scratch per task.
- Models did not share representations across tasks.
- Domain shift required full retraining.

After pre-training

- A small labelled dataset is often sufficient.
- The model already knows grammar, entities, co-occurrence.
- One backbone serves dozens of downstream tasks.
- Domain adaptation requires only additional fine-tuning.

Illustrative statistic

FinBERT (Yang et al. 2020) achieves state-of-the-art financial sentiment classification using only $\sim$4,800 labelled sentences. Training a comparable model from scratch would require orders of magnitude more labelled data.

BERT and the Encoder Family

Introduction to BERT

BERT – Bidirectional Encoder Representations from Transformers
Devlin, Chang, Lee, Toutanova (Google AI, 2018)

- First model to pre-train a *deep bidirectional* transformer using a masked language modelling objective.
- Encoder-only: no causal mask – every token attends to every other token in both directions simultaneously.
- Achieved new state-of-the-art on 11 NLP benchmarks upon release.
- The defining pre-trained model of the 2019–2021 period; still the most widely deployed class of models for classification and extraction tasks.

Central architectural decision

Bidirectionality is ideal for *understanding* tasks (sentiment, NER, span extraction). Its consequence – inability to generate text autoregressively – is a feature, not a bug, for those use cases.

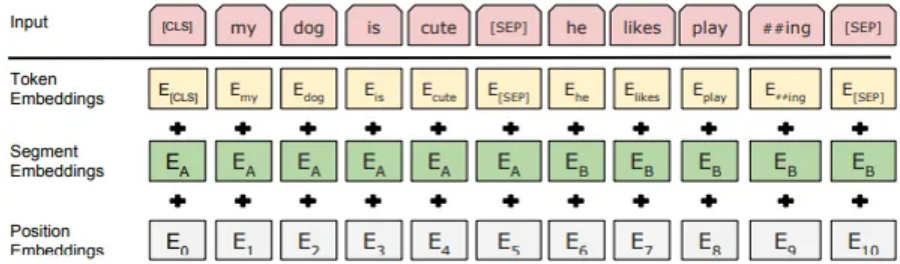
BERT Input Representation

For a sequence of tokens $x_1, \ldots, x_n$, the input to BERT is the elementwise sum of three embedding vectors:

$$\mathbf{e}_i = \underbrace{\mathbf{e}_i^{\text{tok}}}_{\text{token emb.}} + \underbrace{\mathbf{e}_i^{\text{seg}}}_{\text{segment emb.}} + \underbrace{\mathbf{e}_i^{\text{pos}}}_{\text{position emb.}}$$

Embedding	**What it encodes**
Token	Identity of the WordPiece subword unit
Segment	Sentence A or sentence B (for sentence-pair inputs)
Position	Absolute position in the sequence (*learned*, not sinusoidal)

BERT Input Representation



Note on position embeddings

Unlike the original transformer (Chapter 4, sinusoidal), BERT uses *learned* position embeddings. Maximum sequence length is therefore hard-capped at 512 tokens.

WordPiece Tokenization

BERT uses **WordPiece** tokenization: an iterative subword-merging algorithm (closely related to BPE from Chapter 1). Continuation subwords are prefixed with ##:

$$\texttt{playing} \rightarrow \texttt{play} + \texttt{\#\#ing}$$

Financial examples

Input	**WordPiece tokens**
non-performing	non, -, performing
EBITDA	EB, ##IT, ##DA
$12,345.67	$, 12, ,, 345, ., 67
repo (repurchase agreement)	repo (single token)

Special Tokens: [CLS] and [SEP]

[CLS] – Classification token

- Prepended to every input sequence.
- No intrinsic meaning at initialisation; shaped entirely by training.
- After pre-training, its final hidden state aggregates sentence-level information.
- Used as input to the classification head for sentence-level tasks.

[SEP] – Separator token

- Appended at the end of each sentence.
- For sentence-pair inputs, separates A from B.

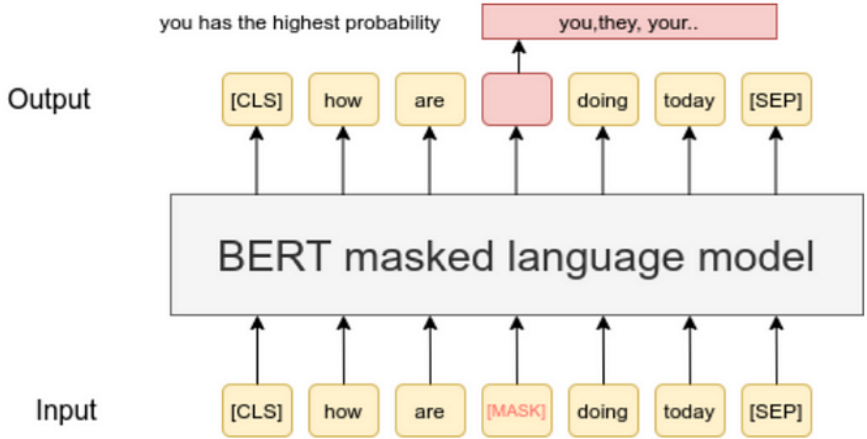
Sentence-pair example:

```
[CLS] the yield rose [SEP]
inflation fears drove the move [SEP]
```

Key insight

The [CLS] output is BERT's understanding *at the sentence level*: a single 768-dimensional vector summarising the entire input. For classification, only this vector is fed to the downstream head – all other token representations are discarded.

Pre-training Objective 1: Masked Language Modelling



Pre-training Objective 1: Masked Language Modelling

MLM procedure: select 15% of input tokens for prediction. For each selected token, apply one of three treatments:

Share	Treatment	Example
80%	Replace with `[MASK]`	"The yield `[MASK]` sharply."
10%	Replace with a random token	"The yield *river* sharply."
10%	Leave unchanged	"The yield *rose* sharply."

Objective: predict the original token at each selected position.

Why the 80/10/10 split?

If only `[MASK]` were used, pre-training would be too different from inference, where no mask tokens appear. The random and unchanged cases reduce this mismatch and force the encoder to build useful representations for all token positions.

Why MLM Forces Bidirectionality

Consider:

"The central bank `[MASK]` interest rates by 50 basis points."

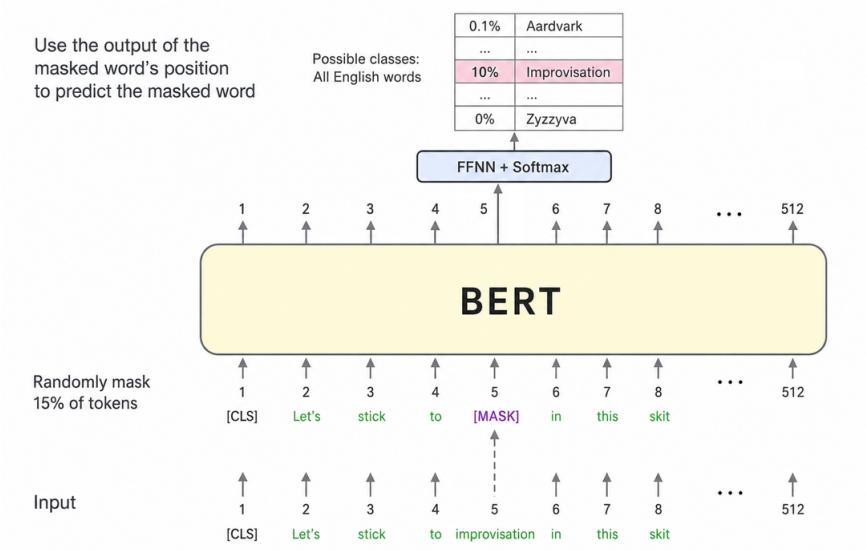
Predicting the masked word (*raised* / *cut* / *held*) requires reading *both* left and right context simultaneously.

GPT cannot do this: the causal mask (Chapter 4) blocks right-to-left attention. Filling a mask with a unidirectional model requires a separate bi-directional mechanism.

Connection to Chapter 4

MLM is the *training signal* that makes bidirectional self-attention (no causal mask) maximally useful. The masking is applied to the *loss*, not to the attention matrix. The encoder attends freely in both directions on every forward pass.

Pre-training Objective 1: Masked Language Modelling



Pre-training Objective 2: Next Sentence Prediction

NSP task: given a token pair [CLS] A [SEP] B [SEP], predict whether B is the actual next sentence following A (IsNext) or a randomly sampled sentence (NotNext).

- 50 % positive pairs: consecutive sentences from the corpus.
- 50 % negative pairs: B drawn from a different document.
- Prediction made from the [CLS] final hidden state.

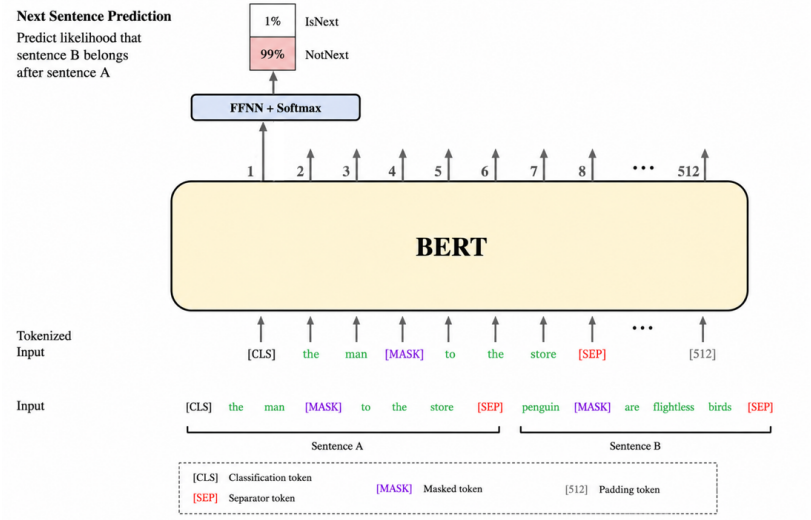
An honest assessment: NSP is largely useless

RoBERTa (Liu et al., 2019) showed that *removing NSP* and training on MLM alone with larger batches and more data produces a consistently better model on all downstream benchmarks.

Probable reason: random sentence pairs are trivially distinguishable without deep inter-sentence reasoning. The task is too easy.

Methodological lesson: ablation studies matter. Intuitive pre-training objectives are not necessarily beneficial.

Pre-training Objective 2: Next Sentence Prediction



BERT Architecture: Base, Large, and Reference

Model	Encoder layers	d_{model}	Heads	Parameters
Transformer (Vaswani 2017)	6	512	8	≈65 M
BERT-Base	12	768	12	110 M
BERT-Large	24	1024	16	340 M
DistilBERT	6	768	12	66 M

- BERT-Base was designed to match the compute of the original transformer for fair comparison.
- BERT-Large significantly outperforms Base at greater compute cost.
- **DistilBERT** (Sanh et al., 2019): 40 % smaller, 60 % faster, retains 97 % of BERT-Base performance. Preferred for latency-sensitive financial deployments.

From Pre-training to Fine-tuning

Fine-tuning recipe:

1. **Initialise** all encoder weights from the pre-trained checkpoint.
2. **Add a task-specific head**: a linear layer (or small MLP) on top of the `[CLS]` vector for classification, or on top of each token vector for NER and span extraction.
3. **Train end-to-end** on the labelled dataset with a small learning rate (typically $2\text{–}5 \times 10^{-5}$).

Partial freezing

Freeze lower encoder layers; update only the top layers and head. Faster; useful when the labelled dataset is very small.

Full fine-tuning

Update all parameters. Achieves best performance when sufficient labelled data is available ($\gtrsim$1,000 examples).

Fine-tuning for Classification: The [CLS] Head

For sentence or document classification:

$$\hat{y} \;=\; \mathrm{softmax}\big(W_c\,\mathbf{h}_{\texttt{[CLS]}} + \mathbf{b}_c\big)$$

where $\mathbf{h}_{\texttt{[CLS]}} \in \mathbb{R}^d$ is the final hidden state of [CLS] and $W_c \in \mathbb{R}^{K\times d}$ is the *only new parameter* to be learned.

Financial example: earnings call sentiment

Input: *"Revenue grew 12 % year-over-year, driven by strong demand in cloud services."*
Label: positive

This is exactly the task targeted by FinBERT. The entire 110 M-parameter model is fine-tuned on ≈4,800 labelled sentences – feasible only because of pre-training.

Fine-tuning for Token-Level Tasks: NER

For named entity recognition or extractive question answering, *each token position* receives an independent prediction:

$$\hat{y}_i \;=\; \mathrm{softmax}\big(W_t\,\mathbf{h}_i + \mathbf{b}_t\big), \quad i = 1, \ldots, n$$

Same pre-trained backbone; different output head.

Financial NER

- Extract company names, tickers, dates, and monetary figures from SEC filings.
- Label risk factors in 10-K documents by type.
- Example: "Apple reported revenue of $117 B in Q4 2023."

Note

A single BERT checkpoint can be fine-tuned separately for classification (sentence-level head) and NER (token-level head), yielding two task-specific models from one pre-trained backbone.

Domain Mismatch Revisited

In Chapter 2 we established that general-purpose embeddings misrepresent financial terminology. The same problem afflicts BERT:

Term	**General nearest neighbour**	**Financial nearest neighbour**
bond	James Bond, spy, film	yield, coupon, maturity
call	phone, ring, dial	option, strike, put
alpha	Greek, beta, alphabet	excess return, benchmark
repo	repository, git, code	repurchase, overnight, collateral

FinBERT: Domain-Specific Pre-training

FinBERT (Yang, UY, Huang – IJCAI 2020): continue pre-training BERT-Base on a large financial corpus *before* task-specific fine-tuning.

- **Domain corpus:** Reuters TRC2 financial news, Bloomberg financial news, earnings call transcripts. $\approx$1.8 B tokens of financial text.
- **Procedure:** initialise from BERT-Base; continue MLM on the financial corpus; fine-tune on the target task.
- **Result on Financial PhraseBank:** FinBERT $\approx$88 % vs. BERT $\approx$79 % accuracy.

BERT: Strengths and Hard Limits

Strengths

- Strong on classification and extraction with small labelled datasets.
- Bidirectionality is optimal for understanding.
- Fast, auditable, well-understood.
- Excellent financial variants on Hugging Face.
- DistilBERT for latency-constrained settings.

Hard limits

- **512-token context:** long financial documents (10-K filings) exceed this.
- **Frozen knowledge:** no information after the pre-training cutoff.
- **No external memory:** cannot access updated information without retraining or RAG.

Encoder Family: Summary

Model	Pre-training data	Key change	Main financial use
BERT-Base	Wikipedia + Books	Bidirectional MLM + NSP	General baseline
RoBERTa	More data	No NSP; longer training	Stronger baseline
FinBERT	Financial corpora	Domain adaptation	Sentiment; NER
DistilBERT	BERT teacher model	Smaller and faster	Low-latency production
Sentence-BERT	NLI / sentence pairs	Siamese training	Semantic search; RAG

GPT and the Decoder Family

Autoregressive Language Modelling

The decoder pre-training objective is classical: predict the next token given all previous tokens.

$$\mathcal{L}(\theta) \;=\; -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

- **No annotation required:** the next token *is* the label.
- **Unbounded self-supervised data:** any text corpus is a training set.
- **Causal structure enforced:** position i can only attend to positions $1, \ldots, i$ (the causal mask from Chapter 4, Section IV).

Distinction from n-gram language models

Both model the probability of the next token. The difference is representational depth: GPT learns rich hierarchical contextual representations through transformer self-attention; an n-gram model counts only fixed-length surface patterns.

Autoregressive Generation: The Expanding Window

At inference time:

1. Feed a prompt of n tokens to the model.
2. Obtain a probability distribution over the full vocabulary: $P(x_{n+1} \mid x_1, \ldots, x_n)$.
3. Sample or select one token $\hat{x}_{n+1}$.
4. Append $\hat{x}_{n+1}$ to the sequence. Go to step 1.
5. Repeat until a stopping condition is reached (end-of-sequence token or maximum length).

Cost structure

No single forward pass produces the complete output. Each additional token requires one full forward pass over the growing sequence. Generation cost scales *linearly in output length*. For long financial documents, this is a non-trivial infrastructure concern.

Temperature and Top-p Sampling

The model outputs a logit vector $\mathbf{z} \in \mathbb{R}^{|V|}$. The token probability is:

$$P(x = v \mid \text{context}) = \frac{\exp(z_v / T)}{\sum_{v'} \exp(z_{v'} / T)}$$

$T \to 0$: greedy

Always pick the argmax. Deterministic, focused. Use for factual extraction.

$0 < T < 1$: sharp

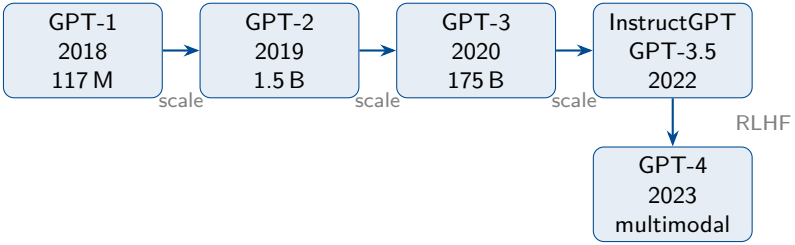
High-prob tokens dominate. Low diversity.

$T > 1$: flat

More randomness. **Never** for outputs that will be acted upon.

Top-p (nucleus) sampling: retain the smallest set of tokens whose cumulative probability exceeds p (typically 0.90–0.95); sample from this nucleus. Avoids rare tokens without the rigidity of top-k.

The GPT Genealogy



Three themes across the timeline:

- **Scale:** parameter count grows by three orders of magnitude.
- **Opacity:** from fully open (GPT-1/2) to closed weights and undisclosed training details (GPT-3.5/4).
- **Alignment:** shift from next-token prediction to instruction-following via RLHF.

GPT-1, GPT-2, and GPT-3

GPT-1 (Radford et al., 2018 – 117 M parameters)
First demonstration: unsupervised pre-training + supervised fine-tuning outperforms task-specific architectures trained from scratch. *Pre-training learns transferable features.*

GPT-2 (2019 – 1.5 B)
Trained on WebText (40 GB, Reddit outlinks). Many tasks can be performed from the prompt alone, with no fine-tuning. OpenAI withheld the full model initially – the first public signal that scale creates unexpected capability surprises.

GPT-3 (Brown et al., 2020 – 175 B)
The defining discovery: **in-context learning**.

- **Zero-shot:** task description in the prompt only.
- **One-shot:** one example input-output pair.
- **Few-shot:** several examples.

Prompt Engineering and Chain-of-Thought

Prompt design has measurable, reproducible effects on output quality.

Chain-of-thought (CoT) prompting (Wei et al., 2022):
ask the model to reason step by step before giving a final answer.

Financial CoT example

Prompt: "Given the balance sheet data below, determine whether the current ratio indicates adequate short-term liquidity. Think step by step."

Model: "Current assets = \$450 M. Current liabilities = \$380 M. Current ratio $= 450/380 \approx 1.18$. A ratio above 1.0 indicates assets exceed liabilities. However, 1.18 is thin for a manufacturing company relative to the industry median of 1.5. . . "

Critical caveat

The reasoning chain is fluent and often correct, but can be wrong at any step. It must be verified against authoritative sources, not accepted as a reliable audit trail.

Scaling Laws and Alignment

Scaling Laws

Kaplan et al. (2020): performance scales as a power law in parameters N, training tokens D, and compute C:

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}$$

Implication: for a fixed compute budget, maximise model size.

Hoffmann et al. (2022) – Chinchilla: the Kaplan law severely *underweights training data*. Optimal scaling requires N and D to grow together:

$$N_{\text{opt}} \propto C^{0.5}, \qquad D_{\text{opt}} \propto C^{0.5}$$

Chinchilla (70 B parameters, 1.4 T tokens) outperforms **Gopher** (280 B, 300 B tokens) on most benchmarks, despite having four times fewer parameters.

Practical implication

Many widely-deployed models are *over-parameterised* relative to their training data. Bigger is not unconditionally better.

Emergent Abilities: What They Are and Why They Are Debated

Wei et al. (2022): certain abilities (multi-step arithmetic, code synthesis) appear to emerge *discontinuously* at threshold scale: absent below a parameter count, present above it.

Schaeffer et al. (2023): emergence may be an artefact of *non-linear evaluation metrics*. With smooth metrics, performance improves continuously. The discontinuity is in the metric, not in the model.

What practitioners should conclude

Emergence is a useful heuristic for capability planning, not an established quantitative law. Do not assume a capability exists without explicit benchmarking.

Financial implication

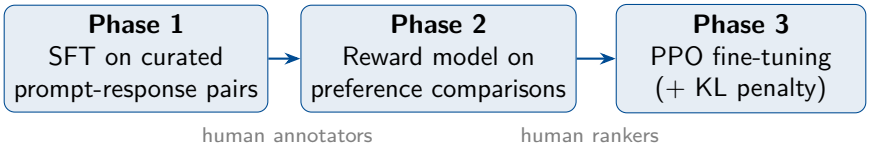
A model that appears to understand financial statements on standard benchmarks may fail systematically on edge cases not present in the evaluation set. **Benchmark on your own data.**

The Alignment Problem and RLHF

With GPT-3, the model follows the *statistical structure* of the prompt. A model trained on internet text replicates its biases, errors, and harmful patterns.
Alignment: making model behaviour consistent with human preferences and values.

InstructGPT and RLHF (Ouyang et al., 2022)

Reinforcement Learning from Human Feedback: train the model to maximise a reward derived from human preferences over which of two responses to the same prompt is better.



The **KL penalty** prevents reward hacking: the fine-tuned model is kept close to the SFT baseline.

DPO: Direct Preference Optimisation

Rafailov et al. (2023): RLHF is complex and unstable. Training a separate reward model introduces variance; PPO is notoriously difficult to tune.

Key insight: the optimal RLHF policy has a closed form in terms of the reference model. Substituting into the preference likelihood yields a *binary cross-entropy loss* on the language model directly:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid c)}{\pi_{\text{ref}}(y_w \mid c)} - \beta \log \frac{\pi_\theta(y_l \mid c)}{\pi_{\text{ref}}(y_l \mid c)}\right)\right]$$

where y_w is the preferred response and y_l the rejected one.

Practical consequence

DPO requires no reward model and no RL training loop. It is now the dominant alignment technique for open-weight models due to its simplicity and stability.

Alignment and Finance: A Risk Management Perspective

Specific scenarios:

- Model invents a regulatory requirement that does not exist; the compliance team acts on it.
- Model fabricates a citation to a non-existent research report; the analyst includes it in a brief.
- Model generates an incorrect numerical summary of a balance sheet; the portfolio manager acts on the figure.

Regulatory context: MiFID II, EU AI Act (2024), and US SR 11-7 create accountability requirements for automated outputs that influence investment or credit decisions. An aligned model reduces risk but is not a substitute for human review.

Open Models and the Transparency Imperative

GPT-4 (2023): multimodal; improved benchmarks. The technical report is 98 pages but discloses almost nothing technical: no architecture, no parameter count, no training data composition.

Consequence for finance

A black-box commercial model cannot satisfy model risk management, audit, or explainability requirements in regulated environments (SR 11-7, EU AI Act). Purchasing from a vendor does not transfer the validation obligation.

Open alternatives

- **LLaMA** (Meta 2023–2024): open weights, competitive, deployable on-premises.
- **Mistral-7B:** strong at small scale; fits on a single GPU.
- **Gemma, Phi, Falcon:** further options with different licence terms.

Decoder Family: Summary

Model	Params	Key innovation	Open weights?
GPT-2	1.5 B	Zero-shot at scale	Yes
GPT-3	175 B	In-context learning	No
InstructGPT	–	RLHF alignment	No
GPT-4	–	Multimodal reasoning	No
LLaMA-2	7–70 B	Open-weight LLM family	Yes, gated
Mistral-7B	7 B	Strong small model	Yes

Retrieval and RAG

The Fundamental Limit: Frozen Knowledge

All pre-trained models encode world knowledge in their **weights** at training time.

What frozen knowledge means

- No knowledge of events after the training cutoff.
- Cannot access updated earnings, rate decisions, or regulatory changes without retraining.
- Fine-tuning on new data is expensive, slow, and risks catastrophic forgetting.

The context window as external memory

An LLM can read and reason over *any text* placed in its context window, even if never seen during training.

RAG exploits this: retrieve relevant documents at query time, inject them into the prompt, generate a grounded answer.

Dense Semantic Search: The Bi-encoder

Recall from Chapter 1: BM25 retrieval uses sparse lexical matching. Dense retrieval uses **semantic similarity in embedding space**.

Bi-encoder architecture:

- Query encoder: $\mathbf{q} = \mathrm{Enc}(q) \in \mathbb{R}^d$
- Document encoder: $\mathbf{d}_i = \mathrm{Enc}(d_i) \in \mathbb{R}^d$
- Retrieval score: $\mathrm{score}(q, d_i) = \dfrac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \, \|\mathbf{d}_i\|}$

Key property: offline pre-computation

Document vectors are pre-computed and stored. Retrieval at query time is a nearest-neighbour search, not a full forward pass over each document. This makes dense retrieval scalable to millions of documents.

Vector Search at Scale: FAISS

Exhaustive cosine search over millions of documents is too slow.
FAISS (Johnson et al., 2019) provides approximate nearest-neighbour search:

- **Inverted File Index (IVF):** partition the vector space into Voronoi cells; at query time, search only the closest cells.
- **Product Quantisation (PQ):** compress vectors into short codes, reducing memory and accelerating distance computation.

Practical numbers

Millisecond-level retrieval over 10 M vectors of dimension 768 on a single GPU. IVF + PQ reduces the index for 1 M documents by 8–32$\times$ relative to storing full float32 vectors.

Vector Search at Scale: FAISS

Managed alternatives

Qdrant, Weaviate, Pinecone, and similar vector database services offer FAISS-style search with additional features (dynamic insertion, metadata filtering, hybrid queries). Choice depends on update frequency, latency requirements, and data residency constraints – the last being a critical concern for financial institutions.

Sparse vs. Dense Retrieval Revisited

From Chapter 1, sparse and dense methods have **complementary failure modes**:

	Sparse retrieval	**Dense retrieval**
Method	BM25 / lexical matching	Bi-encoder embeddings
Strengths	Exact terms; names; numbers	Synonyms; paraphrases
Failure mode	May miss semantic variants	May confuse similar meanings
Example	Misses “rate hike”	Confuses Apple Inc. with fruit
Interpretability	High: term scores visible	Low: opaque vectors

Sparse vs. Dense Retrieval Revisited

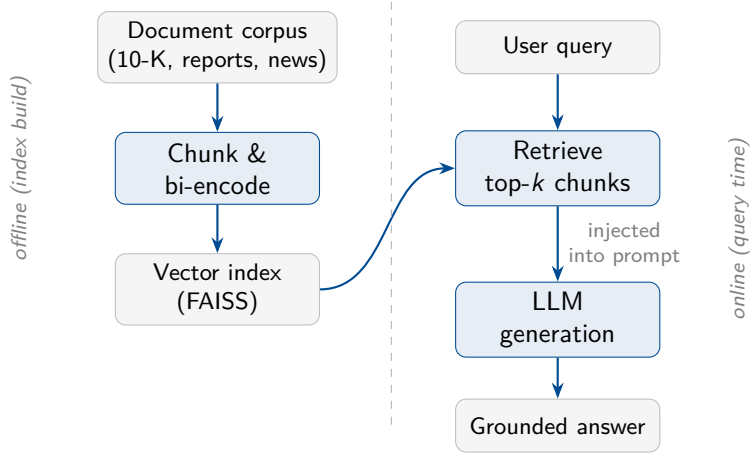
Hybrid retrieval: run both methods in parallel and fuse the two rankings with **Reciprocal Rank Fusion (RRF)**:

$$\mathrm{RRF}(d) = \sum_{r \in \{\mathrm{sparse},\, \mathrm{dense}\}} \frac{1}{k + \mathrm{rank}_r(d)}$$

where d is a document, r indexes the retrieval method, $\mathrm{rank}_r(d)$ is the rank assigned to document d by method r, and k is usually set to 60.

No score calibration is required: RRF combines rankings, not raw scores.

The RAG Pipeline



RAG for Financial Text: A Concrete Example

Task: "What are the principal risk factors in this company's most recent annual report?"

Offline – index build:

1. Chunk the 10-K into 256-token overlapping segments (stride 50).
2. Encode each chunk with a bi-encoder; store in FAISS.

Online – query time:

1. Encode the query; retrieve top-5 chunks by cosine similarity.
2. Construct the prompt:

Prompt structure

```
System: You are a financial analyst assistant. Answer using only the excerpts
provided. If the information is absent, say so.
[CONTEXT]: {chunk1}...{chunk5}
[QUESTION]: What are the principal risk factors?
[ANSWER]:
```

Chunking Strategies and Their Trade-offs

Chunking is a non-trivial engineering decision with measurable effects on retrieval quality.

Strategy	Advantages	Disadvantages
Fixed-size chunks	Simple and uniform	May split sentences or tables
Sentence-aware	Preserves sentence boundaries	Produces uneven lengths
Paragraph-aware	Preserves semantic units	Long paragraphs may overflow
Overlapping chunks	Reduces boundary effects	Adds redundancy to the index

Financial documents require special handling

Tables, footnotes, section headers, and numeric exhibits are often poorly represented by embeddings trained mainly on prose. In production-quality financial RAG systems, structured data usually requires a dedicated extraction pipeline.

RAG Failure Modes

1. **Retrieval failure.** The correct passage is not among the top-k retrieved. Regardless of generation quality, the answer will be wrong.
   Measure: recall@k on a held-out question set.
2. **Context poisoning.** A retrieved passage is partially relevant and introduces misleading context. The model generates an answer consistent with the retrieved text but incorrect overall.
   Measure: faithfulness – does the answer follow from the retrieved context?
3. **Faithfulness failure.** The model ignores the retrieved context and generates a plausible answer from its parametric knowledge alone.
   Measure: compare answer against retrieved passages, not just against a gold standard.

Evaluation framework

RAGAS (Shahul et al., 2023) provides automated metrics for all three failure modes. Human evaluation remains necessary for high-stakes financial deployments.

Hugging Face

The Hugging Face Ecosystem

Hugging Face provides the standard distribution infrastructure for open NLP models:

- **Model Hub:** >500,000 models (2025), filterable by task, language, licence, and framework.
- **`transformers` library:** load and run any Hub model in a few lines of Python.
- **Model cards:** standardised documentation covering training data, evaluation results, intended use, and known limitations.
- **Datasets library:** standardised access to NLP benchmarks and domain-specific corpora.

Financial models on the Hub (selection)

`ProsusAI/finbert` – `yiyanghkust/finbert-tone` – `nickmuchi/finbert-tone-finetuned-finance-topic-classification` `meta-llama/Llama-2-7b-chat-hf` (gated) – `mistralai/Mistral-7B-Instruct-v0.2`

Model Cards as Due Diligence

A model card is not just documentation. For a financial institution, it is the **first step in a model risk management process**.

Key fields to inspect before deployment:

- **Training data:** composition, time period, known gaps.
- **Evaluation:** which datasets, which metrics, reported numbers.
- **Intended use / out-of-scope use:** does your use case match?
- **Known biases and failure modes:** documented weaknesses.
- **Licence:** can you use it commercially? Can you deploy it on-premises?

Red flags

A model with no card, or a card that omits evaluation details, should not be deployed in any production financial workflow without independent validation. "Available on Hugging Face" is not a quality guarantee.

A Minimal Working Example

1. Financial sentiment (FinBERT):

```
from transformers import pipeline
clf = pipeline("text-classification",
               model="ProsusAI/finbert")
clf("Revenue grew 12% driven by strong cloud demand.")
# => [{'label': 'positive', 'score': 0.97}]
```

A Minimal Working Example

2. Sentence similarity (Sentence-BERT):

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("all-MiniLM-L6-v2")
embs = model.encode(["yield curve inversion",
                     "inverted yield curve signals recession"])
print(util.cos_sim(embs[0], embs[1]))   # => 0.87
```

A Minimal Working Example

3. Minimal RAG loop (conceptual):

```
chunks  = retrieve_top_k(query, faiss_index, k=5)
context = "\n".join(chunks)
answer  = llm.generate(
  f"[CONTEXT]: {context}\n[Q]: {query}\n[A]:")
```

Financial Applications and Critical Assessment

A Taxonomy of Financial NLP Tasks

Category	Financial examples	Model family	Evidence
Classification	Sentiment; filing category	Encoder / FinBERT	Strong
Extraction	NER; relations; numbers	Encoder / token model	Strong
Generation	Summaries; Q&A; drafting	Decoder + RAG	Moderate
Retrieval	Semantic search; peer firms	Bi-encoder + FAISS	Moderate

The Benchmarking Discipline

Before deploying an LLM for any task, **establish a baseline**.
The baseline is the simplest reasonable alternative:

- Logistic regression on TF-IDF features.
- A fine-tuned DistilBERT (66 M parameters, much faster than BERT-Base).
- A rule-based system (dictionary lookup, regex extraction).

The discipline in practice

- An LLM that barely outperforms the baseline is not the right tool.
- An LLM that requires 100× more compute to match the baseline is *definitely* not the right tool.
- A baseline that took one afternoon to build is almost always worth building before committing to an LLM pipeline.

This connects directly to the course's methodological philosophy: method-to-problem fit matters more than method novelty.

Known Failure Mode 1: Hallucination

Hallucination: the model generates fluent, confident text that is factually incorrect.

Financial examples:

- Fabricated earnings figure: "the company reported revenue of \$12.4 B in Q3, up 8 % year-on-year" (invented number).
- Non-existent regulatory citation.
- Incorrect credit rating or index constituent.

Mitigation:

- RAG with faithfulness evaluation: ground the model in retrieved authoritative sources.
- Post-hoc numerical verification: extract all numbers from the output; cross-reference against a structured database.
- Human-in-the-loop for any output that influences a decision.

Important

Hallucination rate is not zero for any current model on any task. It must be *measured*, not assumed away.

Known Failure Mode 2: Temporal Blindness

A model trained up to date T has no knowledge of anything after T.

In finance, knowledge has a short shelf life:

- Quarterly earnings, guidance revisions, dividend announcements.
- Regulatory updates, ESG reclassifications.
- Rate cycle changes, sector rotation.
- M&A events, index reconstitutions.

Mitigation:

- RAG with a regularly updated and monitored document index.
- Report the model's training cutoff date in any output that users may act upon.

No mitigation is free

Regular index updates require pipeline infrastructure and quality control. They do not eliminate the problem for documents that have not yet been indexed.

Known Failure Mode 3: Arithmetic Unreliability

LLMs are trained on text. Numerical computation was never their objective.

They can fail at:

- Multi-step arithmetic: $127.3 \times 0.078 - 14.9$.
- Percentage change: $\Delta\% = (X_t - X_{t-1})/X_{t-1}$.
- Summing a column of figures from a financial table.
- Compound interest and present value calculations.

This is not a fixable quirk – it is architectural.

Design principle

Use the LLM for *language understanding*: identify what to compute, extract the relevant figures from prose, interpret the result. Use *deterministic code* for the arithmetic itself. A tool-augmented LLM (function-calling API, Code Interpreter) is the correct architecture for tasks that mix natural language and computation.

Known Failure Mode 4: Spurious Confidence

RLHF-trained models are incentivised to produce *helpful-sounding* responses. This can increase overconfidence: the model asserts wrong facts with the same tone as correct ones.

Calibration evaluation:

- **Expected Calibration Error (ECE):** how well does stated confidence track actual accuracy?
- **Selective prediction:** can the model identify which outputs are likely wrong?

Practical consequence

Any LLM output in a financial workflow should be accompanied by a confidence estimate and an explicit verification step, not a single unqualified assertion. A model that says "I don't know" is more useful than one that says "The answer is X" when X is wrong.

Regulatory and Governance Considerations

EU AI Act (2024)

- LLMs influencing financial decisions likely qualify as *high-risk AI systems*.
- Requirements: documentation, human oversight, conformity assessment, transparency.
- General-purpose AI models face additional transparency obligations above 10^{25} FLOP training compute.

Model Risk Management (SR 11-7 / SS1/23)

- US Fed / UK PRA guidelines require validation, ongoing monitoring, and documentation for all models influencing decisions.
- Scope includes statistical algorithms, ML models, and by extension LLM components.
- Purchasing from a vendor does not transfer the validation obligation.

Regulatory and Governance Considerations

Key message

Regulatory compliance requires model transparency, validation, and audit trails. A model card is a starting point, not a substitute for internal model risk management.

What the Evidence Shows

Documented production successes

- Financial sentiment classification at scale.
- Named entity recognition in SEC filings.
- Earnings call summarisation and search.
- Compliance document classification.
- Internal chatbots for regulatory Q&A.

Claims that exceed evidence

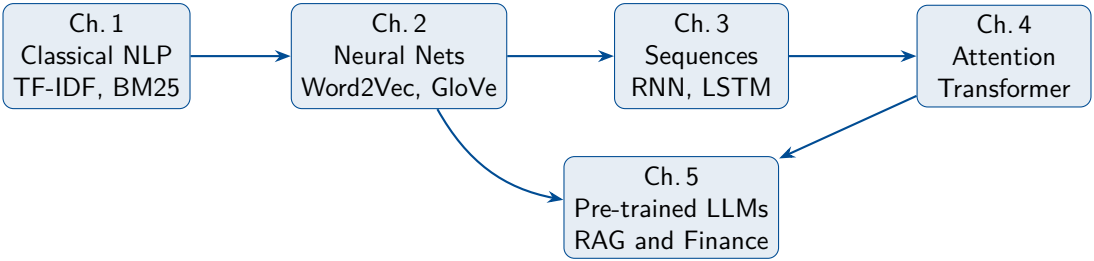
- Autonomous alpha generation from LLM text signals.
- Macro-economic forecasting from news summaries.
- Reliable credit risk assessment from unstructured text.
- Financial reasoning without programmatic verification.

Conclusion and References

Chapter 5 – Key Takeaways

1. The **pre-training / fine-tuning paradigm** decouples language knowledge (from scale, no labels) from task adaptation (small labelled datasets).
2. **BERT** uses MLM to train a deep bidirectional encoder; the `[CLS]` representation enables classification with a minimal task-specific head.
3. **Domain mismatch** is real: financial terminology requires domain-specific pre-training (FinBERT) or fine-tuning on domain-labelled data.
4. **GPT / autoregressive models** generate text token by token; temperature and top-p control stochasticity.
5. **RLHF / DPO** make models follow instructions; alignment is a financial risk management concern, not only ethical.
6. **RAG** extends LLM memory via retrieval; it mitigates but does not eliminate hallucination, temporal blindness, or faithfulness failures.
7. The **benchmarking discipline**: always measure against the simplest reasonable alternative before committing to an LLM solution.

The Full Course Arc



The guiding question of Chapter 1 – finally answered

How does a language system represent, compare, retrieve, and generate text?

Represent: Ch. 1–4 (sparse $\rightarrow$ dense $\rightarrow$ contextual).
Compare: Ch. 1 (BM25, cosine), Ch. 5 (bi-encoder).
Retrieve: Ch. 5 (FAISS, hybrid, RAG).
Generate: Ch. 4–5 (transformer decoder, GPT, LLMs).

References

- Devlin et al. (2019). BERT. *NAACL-HLT*.
- Liu et al. (2019). RoBERTa. *arXiv:1907.11692*.
- Yang et al. (2020). FinBERT. *IJCAI*.
- Reimers & Gurevych (2019). Sentence-BERT. *EMNLP*.
- Brown et al. (2020). GPT-3. *NeurIPS*.
- Ouyang et al. (2022). InstructGPT / RLHF. *NeurIPS*.
- Hoffmann et al. (2022). Chinchilla. *NeurIPS*.
- Rafailov et al. (2023). DPO. *NeurIPS*.
- Wei et al. (2022). Emergent Abilities. *TMLR*.
- Schaeffer et al. (2023). Are Emergent Abilities a Mirage? *NeurIPS*.
- Lewis et al. (2020). RAG. *NeurIPS*.
- Johnson et al. (2019). FAISS. *IEEE Trans. Big Data*.
- Shahul et al. (2023). RAGAS. *arXiv:2309.15217*.
- Sanh et al. (2020). DistilBERT. *EMC^2 NeurIPS Workshop*.